Producer-Consumer

Season: 5 · Concurrency · Category: Concurrency · Level: Intermediate

Intent

- Decouple producers and consumers via a bounded buffer.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Assembly line with a parts bin.

When to Use

- Concurrent / parallel workloads.
- The design force named in Intent shows up more than once.

When Not to Use

- Pure CPU-bound work already parallelized.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.concurrency.producerconsumer;

import java.util.ArrayDeque;
import java.util.Queue;

public final class BoundedQueue<T> {
    private final Queue<T> q = new ArrayDeque<>();
    private final int capacity;

    public BoundedQueue(int capacity) { this.capacity = capacity; }

    public synchronized void put(T item) throws InterruptedException {
        while (q.size() == capacity) wait();
        q.add(item);
        notifyAll();
    }

    public synchronized T take() throws InterruptedException {
        while (q.isEmpty()) wait();
        var item = q.remove();
        notifyAll();
        return item;
    }

    public synchronized int size() { return q.size(); }
}
```

Modern Java 25 / Functional

- Executors.newVirtualThreadPerTaskExecutor() is the modern default for blocking tasks.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: concurrency/producerconsumer/
```

Pitfalls

- Prefer java.util.concurrent and virtual threads over custom pools.
- Always interrupt/shutdown cleanly.

Interview Q&A

- Q: Virtual threads vs platform threads? A: Virtual threads excel at blocking I/O concurrency; still bound by CPU for pure compute.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Decouple producers and consumers via a bounded buffer.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Assembly line with a parts bin. [0:40] Intent — State the design force in one sentence: Decouple pro...